Aula 6 - Retrieval-Augmented Generation (RAG)

Geração assistida por recuperação de contexto

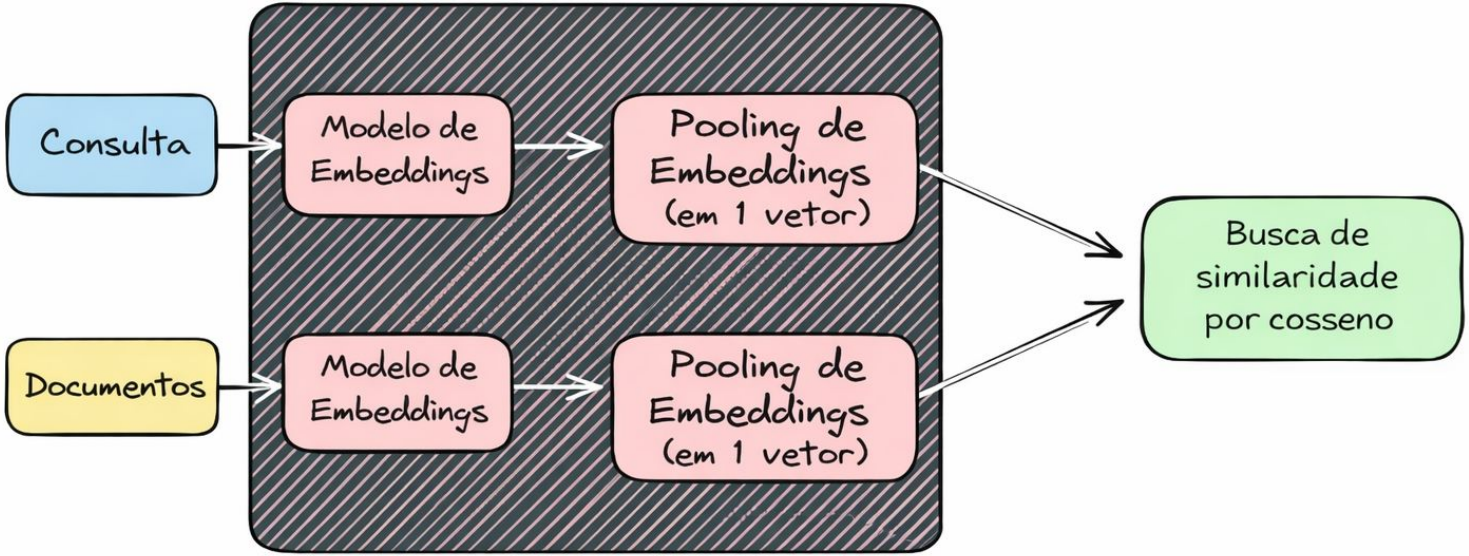
Prof. Marcelo Keese Albertini
Universidade Federal de Uberlândia

Objetivo

- Queremos prover informações atualizadas para LLMs.
- Opções
  - Re-treino (finetune) - $$$$$
  - Retrieval-Augmented Generation (**RAG**)

Abordagem simples: modelos “bi-encoders”

- **Documentos**/sentenças são encodados sozinhos, **sem contexto**
- **Consultas** são encodadas sozinhas, **sem contexto**



https://pendu.github.io/Advanced-Retreival-and-Re-ranking/_downloads/429c068af4e2fc26002cf7f456ced88c/ben_claive_beyond_basics_of_rag_talk.pdf

Exemplo: modelo Bi-Encoder

```
from sentence_transformers import SentenceTransformer
from sklearn.metrics.pairwise import cosine_similarity

model = SentenceTransformer("all-MiniLM-L6-v2")

query_emb = model.encode("Como funciona energia solar?")
doc_emb = model.encode("A energia solar é luz do sol em eletricidade por meio de células fotovoltaicas.")

score = cosine_similarity([query_emb], [doc_emb])
print(score)
```

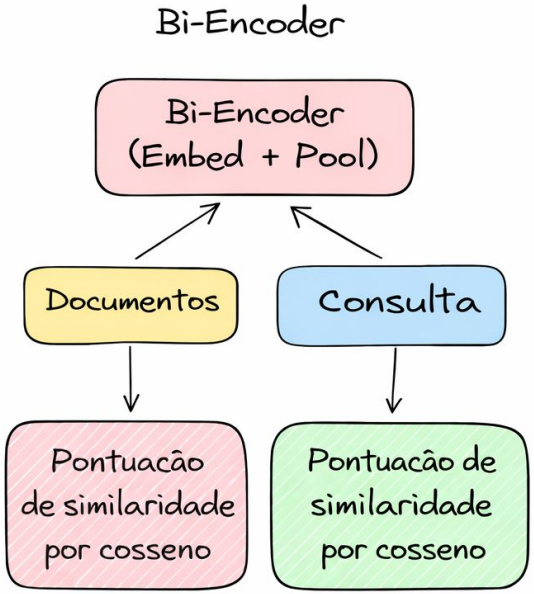
Calcula similaridade **fora** do modelo de embedding.

Implementação simples (Só Bi-encoder)

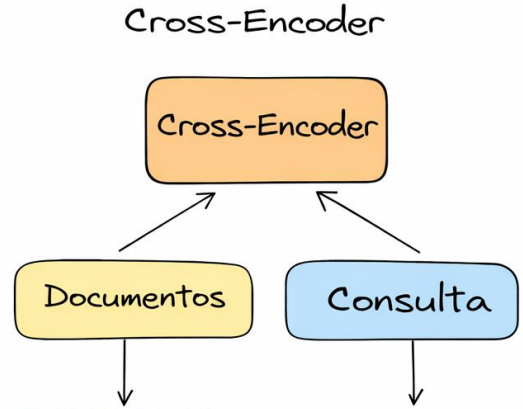
```python
from sentence_transformers import SentenceTransformer
import numpy as np

# Load embedding model
modelo = SentenceTransformer("all-MiniLM-L6-v2")

# Embed your documents (do this once, store the results)
documentos = ["Brasil é grande.", "França come queijo.", "Brasileiro é feliz."]
doc_embeddings = model.encode(documentos, normalize_embeddings=True)

# At query time: embed query and find similar documents
consulta = "Me conte sobre o Brasil."
consulta_embedding = model.encode(consulta, normalize_embeddings=True)

# Compute similarities (this IS your "vector database" at small scale)
similaridades = np.dot(doc_embeddings, consulta_embedding.T)
top_k_indices = np.argsort(similaridades)[-3:][::-1]

resultados = [documentos[i] for i in top_k_indices]
print(resultados)
```

```
['Brasil é grande.', 'Brasileiro é feliz.', 'França come queijo.']
```

Outro tipos de modelo de Embedding: Cross-Encoder



sentence-transformers/all-MiniLM-L6-v2

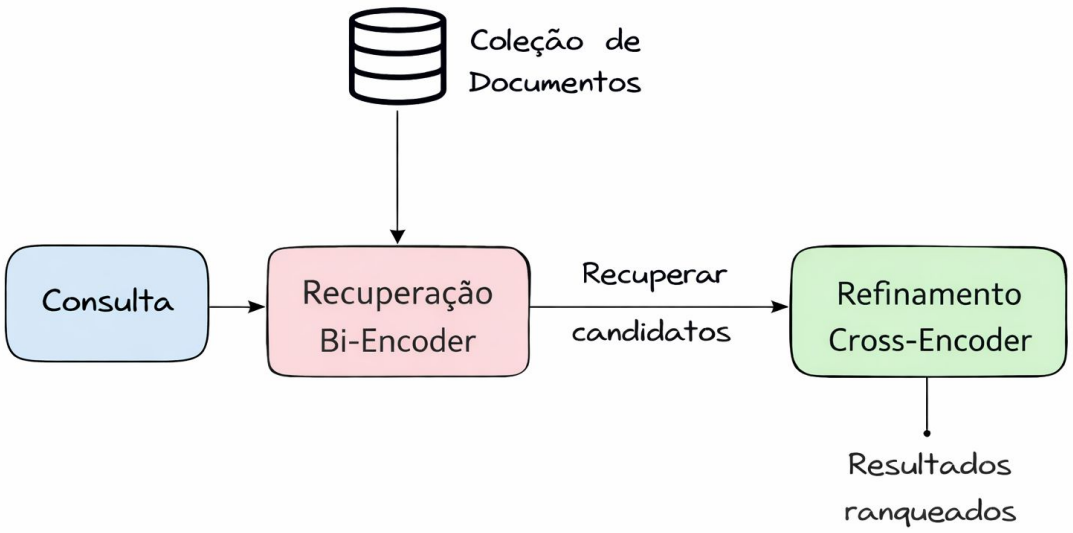


Efetivamente, calcula probabilidade de documento atender à consulta.

cross-encoder/ms-marco-MiniLM-L-6-v2

Refinamento com Cross-Encoder

- Modelo cross-encoder obtém embedding para pares “Consulta, Documento”
- Caro, mas melhor qualidade



Exemplo: modelo Cross-Encoder

```python
from sentence_transformers import CrossEncoder

modelo = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
consulta = "O que é energia solar?"
documento = "Energia solar é luz do sol em eletricidade."
score = modelo.predict([(consulta, documento)])
print(score)

tokenizer = AutoTokenizer.from_pretrained("cross-encoder/ms-marco-MiniLM-L-6-v2")
x = tokenizer(consulta, documento, return_tensors="pt", padding=False, truncation=True)

tokens = tokenizer.convert_ids_to_tokens(x["input_ids"][0])
token_types     = x["token_type_ids"][0].tolist()

# Mostrar token + tipo (0 = query, 1 = doc)
for t, seg in zip(tokens, token_types):
   print(f"{t:>12}  segmento={seg}")
```

```
   [CLS]  segmento=0
       o  segmento=0
     que  segmento=0
       e  segmento=0
      en  segmento=0
    ##er  segmento=0
   ##gia  segmento=0
   solar  segmento=0
       ?  segmento=0
   [SEP]  segmento=0
      en  segmento=1
    ##er  segmento=1
   ##gia  segmento=1
   solar  segmento=1
       e  segmento=1
     luz  segmento=1
      do  segmento=1
     sol  segmento=1
      em  segmento=1
      el  segmento=1
    ##et  segmento=1
   ##ric  segmento=1
  ##idad  segmento=1
     ##e  segmento=1
       .  segmento=1
   [SEP]  segmento=1
```

Implementação simples (Bi-encoder + Cross-Encoder)

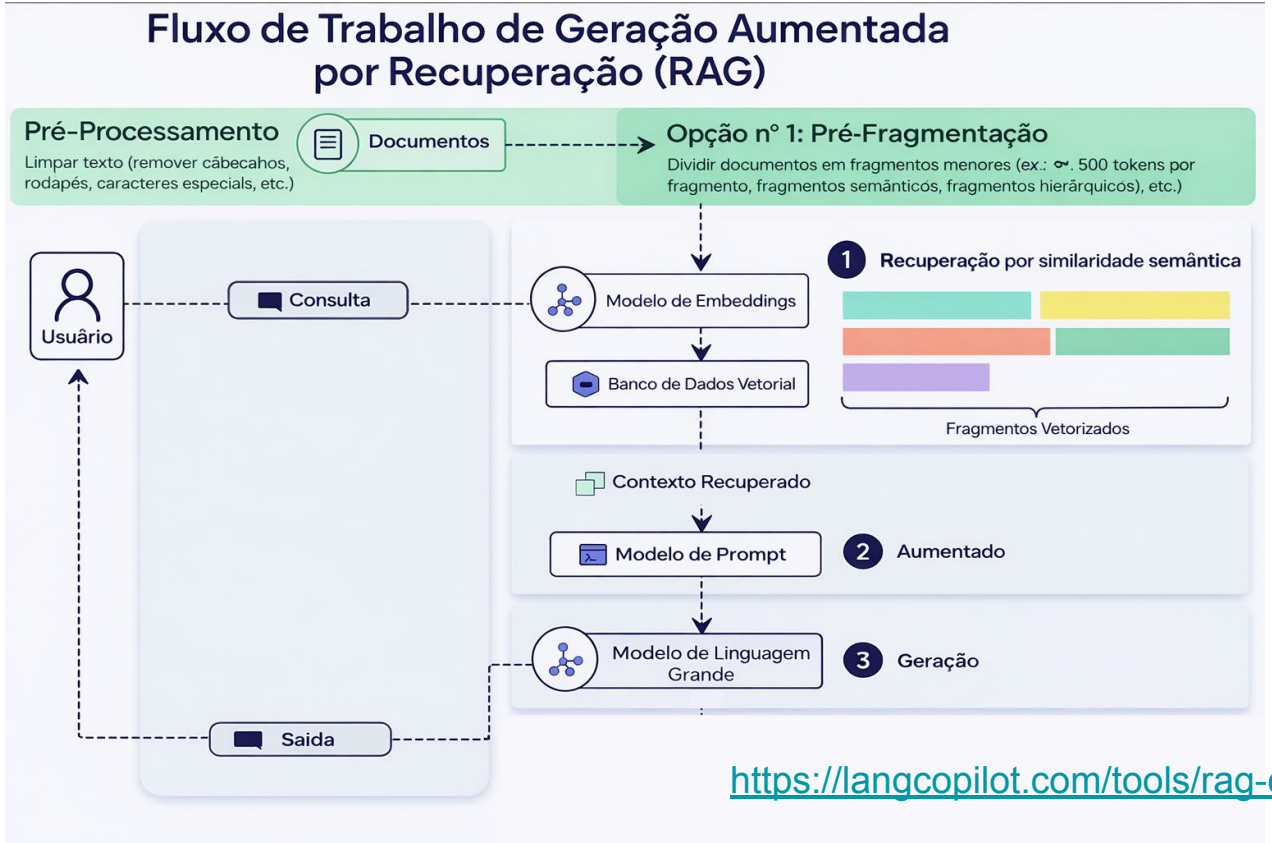
```
from sentence_transformers import SentenceTransformer, CrossEncoder; import numpy
modelo_bi_encoder = SentenceTransformer("all-MiniLM-L6-v2")

documentos = ["Brasil é grande.", "França come queijo.", "Brasileiro é feliz.", "Brasa é quente"]
doc_embeddings = modelo_bi_encoder.encode(documentos, normalize_embeddings=True)
consulta = "Me conte sobre o Brasil."
consulta_embedding = modelo_bi_encoder.encode(consulta, normalize_embeddings=True)
similaridades = numpy.dot(doc_embeddings, consulta_embedding.T)
documentos_filtrados = [documentos[i] for i in numpy.argsort(similaridades)[-3:][::-1]]  # top-K (K=3)
print("Documentos filtrados: ", documentos_filtrados)
```

```
Documentos filtrados:  ['Brasil é grande.', 'Brasileiro é feliz.', 'Brasa é quente']
```

```
modelo_cross_encoder = CrossEncoder("cross-encoder/ms-marco-MiniLM-L-6-v2")
# Obtém as pontuações de relevância do Cross-Encoder
scores = modelo_cross_encoder.predict([[consulta, s] for s in documentos_filtrados])
documentos_rerankeados = sorted(zip(scores, documentos_filtrados), key=lambda x: x[0], reverse=True)
print("Reranking:", documentos_rerankeados)
```

```
Reranking: [(np.float32(-2.804567), 'Brasil é grande.'),
(np.float32(-2.8706286), 'Brasileiro é feliz.'),
(np.float32(-10.317237), 'Brasa é quente')]
```

Chunking (segmentação de documentos longos)



https://langcopilot.com/tools/rag-chunk-lab

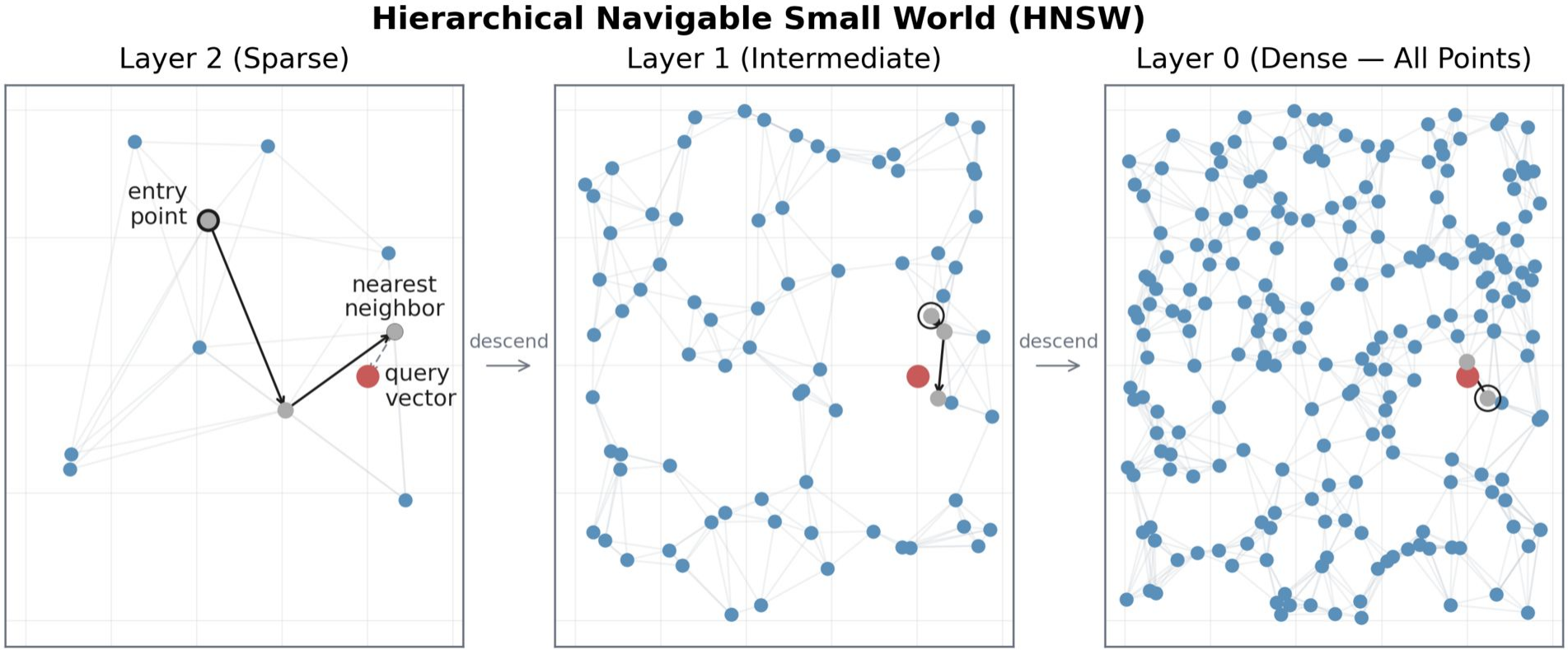
https://weaviate.io/blog/chunking-strategies-for-rag

https://chunkviz.up.railway.app/

Chunking - exemplos

https://langcopilot.com/tools/rag-chunk-lab

Vector DBs: Hierarchical Navigable Small World (HNSW)



Busca híbrida com BM25

```python
from rank_bm25 import BM25Okapi
from sentence_transformers import SentenceTransformer; import numpy

tokenized_docs = [doc.lower().split() for doc in documentos]
bm25 = BM25Okapi(tokenized_docs) # Prepare BM25 - busca esparsa

bi_encoder = SentenceTransformer("all-MiniLM-L6-v2")
doc_embeddings = bi_encoder.encode(documentos, normalize_embeddings=True) # busca densa

def hybrid_search(query, doc_embd, docs, top_k=100, alpha=0.5):
    tokenized_query = query.lower().split()
    bm25_scores = bm25.get_scores(tokenized_query)
    bm25_scores = (bm25_scores - bm25_scores.min()) / (bm25_scores.max() - bm25_scores.min() + 1e-6)

    query_emb = bi_encoder.encode(query, normalize_embeddings=True)
    dense_scores = np.dot(doc_embd, query_emb.T).flatten()
    dense_scores = (dense_scores - dense_scores.min()) / (dense_scores.max() - dense_scores.min() + 1e-6)

    hybrid_scores = alpha * dense_scores + (1 - alpha) * bm25_scores
    top_indices = np.argsort(hybrid_scores)[-top_k:][::-1]
    return [docs[i] for i in top_indices]

print(hybrid_search("Brasil", doc_embeddings, documentos))
```

Referências

- https://github.com/answerdotai/rerankers
- https://pendu.github.io/Advanced-Retreival-and-Re-ranking/rag_practical_guide.html
- https://pendu.github.io/Advanced-Retreival-and-Re-ranking/rag_complexity.html
- https://en.wikipedia.org/wiki/Hierarchical_navigable_small_world
- https://community.databricks.com/t5/technical-blog/the-ultimate-guide-to-chunking-strategies-for-rag-applications
- https://builder.aws.com/content/2j9KouvnFz4zVdtsXcbMpN3G6Cw/genai-under-the-hood-part-8-a-visual-guide-to-advanced-chunking-strategies